claude-windows / 577fd4df-ed3f-43d3-854d-d63d732e2e07

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\577fd4df-ed3f-43d3-854d-d63d732e2e07\subagents\workflows\wf_e5e25aeb-1c0\agent-aa54661c14457605f.jsonl`
- SHA-256: `b6e0ea674054fcf1fb0927659af3ba8a423fd578457ea89ac5f30735e76bff6b`
- Source modified: `2026-06-20T16:37:26+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_e5e25aeb-1c0`
- session_id: `577fd4df-ed3f-43d3-854d-d63d732e2e07`

Transcript

1. user (2026-06-20T16:36:52.805Z)

```
You are reviewing a NEW feature (P3) in a TypeScript SPA: a DETERMINISTIC, dependency-free
query compiler that turns plain text ("over 10s", "top 15 longest", "at least 8 shots") into a
filter+sort+limit selection over detected badminton rallies. NO LLM, NO network.

Read these files with the Read tool before judging:
- CORE parser:        C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts
- its unit tests:     C:/Users/avidu/Projects/khelsutra/web/src/lib/query.test.ts
- QueryBar component: C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx
- App wiring:         C:/Users/avidu/Projects/khelsutra/web/src/App.tsx
- selection hook:     C:/Users/avidu/Projects/khelsutra/web/src/hooks/useRallySelection.ts
- RallyGrid:          C:/Users/avidu/Projects/khelsutra/web/src/components/RallyGrid.tsx
- engine data types:  C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts
- the DESIGN CONTRACT: C:/Users/avidu/Projects/khelsutra/docs/PER_RALLY_SELECTION.md  (read §2
decisions D5, §3 honest-vs-fabricated fields, §4.3 "Dead-ends / do-NOT", §9 DoD)

Report ONLY concrete, defensible findings with exact evidence (an input string ? the wrong
parsed
output, or a code line). Do NOT invent issues to fill a quota; an empty findings list is a
valid,
respected answer. Prefer a few high-confidence findings over many speculative ones.

Independently VERIFY this single finding by reading the actual code (do not trust the claim).
Reproduce it concretely ? trace the exact input through the real regexes/logic, or point to the
exact line. Set is_real=true ONLY if you confirm it. If it's a false positive or already
handled,
set is_real=false and explain why. Be a skeptic; default to is_real=false when uncertain.

FINDING (dimension reuse):
{
  "title": "Test gap: summarize() only tested on the 'top' limit path, hiding the first/last
mislabel",
  "severity": "minor",
  "file": "C:/Users/avidu/Projects/khelsutra/web/src/lib/query.test.ts",
  "evidence": "The only summarize assertion with a limit is line 209:
`expect(summarize(parseQuery(\"over 10s top 5\"))).toEqual([\"length > 10s\", \"longest first\",
\"top 5\"])` ? exclusively the duration-desc/\"top\" case. No summarize test covers \"first N\",
\"last N\", or \"N shortest\", which is exactly why the mislabel in finding #1 (\"first 10\" ?
pill \"top 10\") is not caught.",
  "why": "summarize() feeds the user-facing pill; its label correctness across all limit/sort
origins is part of the DoD (\"shows a visible parsed-pill\"). The current coverage gives false
confidence that the pill is honest for every query shape.",
  "suggested_fix": "After fixing finding #1, add summarize cases for \"first 10\" and \"last 5\"
(and \"5 shortest\") asserting the corrected, non-\"top\" labels.",
  "confidence": "high"
```

```
}
```

2. assistant (2026-06-20T16:37:00.973Z)

```
I'll verify this finding by reading the actual code, focusing on the test file's summarize
coverage and the parser's labeling logic.
```

3. user (2026-06-20T16:37:01.908Z)

```
1       // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
2       // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable
axes the
3       // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4       // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5       //
6       // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7       // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8       // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9       import type { RallySegment } from "../api/types";
10
11      export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12      export type FilterField = "duration" | "shot_count";
13      /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14      export interface Filter {
15        field: FilterField;
16        op: CompareOp;
17        value: number; // seconds for duration; a raw count for shot_count
18      }
19
20      export type SortKey = "duration" | "start_time" | "shot_count";
21      export interface Sort {
22        key: SortKey;
23        dir: "asc" | "desc";
24      }
25
26      /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27      export type UnavailableKind = "shot-type" | "player" | "score";
28      export interface Unavailable {
29        token: string;
30        kind: UnavailableKind;
31      }
32
33      export interface ParsedQuery {
34        raw: string;
35        /** ANDed together. */
36        filters: Filter[];
37        sort?: Sort;
38        /** "top N" / "first N" / "N longest". */
39        limit?: number;
40        /** Concepts we understood but cannot honour (shot-type / player / score). */
41        unavailable: Unavailable[];
42        /** True when at least one actionable clause (filter | sort | limit) was understood.
*/
43        recognized: boolean;
44      }
45
46      export interface QueryResult {
47        /** ALL rallies in display order (sorted if a sort was given, else input order). */
48        ordered: RallySegment[];
```

```
49        /** Ids matching the filters (then limited), in display order. */
50        selectedIds: number[];
51      }
52
53      // ?? grammar fragments ????????????????????????????????????????????????????
54      const NUM = "(\\d+(?:\\.\\d+)?)";
55      // Optional trailing unit/noun. "shots?" is listed FIRST so it can't be eaten by the
bare "s"
56      // time-unit alternative; \b stops "m" matching "minutes" etc.
57      const UNIT = "(?:\\s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)\\b)?";
58
59      interface OpPattern {
60        op: CompareOp;
61        re: RegExp;
62        /** Which capture group holds the number / the unit. */
63        numAt: number;
64        unitAt: number;
65      }
66
67      // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
their
68      // bare counterparts ("more than" / "less than") so the negated phrase is consumed
first.
69      function lead(alt: string, op: CompareOp): OpPattern {
70        return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}`, "g"), numAt: 1, unitAt: 2
};
71      }
72      const LEADING: OpPattern[] = [
73        lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
74        lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),
75        lead("exactly|equal to|equals", "="),
76        lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
77        lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
78      ];
79
80      // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
or the
81      // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
present.
82      interface TrailPattern {
83        op: CompareOp;
84        re: RegExp;
85      }
86      const POST_NOUN = "(?:\\s*(shots?)\\b)?";
87      const TRAILING: TrailPattern[] = [
88        { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater))${POST_NOUN}`, "g") },
89        { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
(?:less|fewer|shorter|under|below)${POST_NOUN}`, "g") },
90      ];
91
92      function toSeconds(n: number, unit?: string): number {
93        return unit && /^m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
94      }
95
96      function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
97        const n = Number(numStr);
98        const isShots = !!unit && /^shots?$/.test(unit);
99        return isShots
100         ? { field: "shot_count", op, value: n }
101         : { field: "duration", op, value: toSeconds(n, unit) };
102     }
103
104     const SORT_PATTERNS: Array<[RegExp, Sort]> = [
105       [/\b(?:longest(?: first)?|longer first|by (?:duration|length)|duration desc)\b/, {
```

```
key: "duration", dir: "desc" }],
106       [/\b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
"asc" }],
107       [/\b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
dir: "desc" }],
108       [/\b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
"start_time", dir: "asc" }],
109       [/\b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
desc)\b/, { key: "shot_count", dir: "desc" }],
110       [/\b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
"shot_count", dir: "asc" }],
111     ];
112
113     // Order before kind so a token claimed by an earlier kind isn't double-counted.
114     const UNAVAILABLE_PATTERNS: Array<[UnavailableKind, RegExp]> = [
115       ["shot-type", /\b(smash(?:es)?|clears?|drops?|dropshots?|net
?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?)\b/g],
116       ["player", /\b(players?|opponents?|servers?|receivers?)\b/g],
117       ["score", /\b(points?|score|deuce|game point|set point|match point)\b/g],
118     ];
119
120     // ?? public API ????????????????????????????????????????????????????????????
121
122     export function parseQuery(raw: string): ParsedQuery {
123       const normalized = raw
124         .toLowerCase()
125         .replace(/?/g, ">=")
126         .replace(/?/g, "<=")
127         .replace(/,/g, " ")
128         .replace(/\s+/g, " ")
129         .trim();
130
131       const filters: Filter[] = [];
132       let working = normalized;
133
134       // Extract comparator clauses, blanking each match so later passes can't re-read its
number.
135       for (const pat of LEADING) {
136         working = working.replace(pat.re, (full, ...g) => {
137           filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
138           return " ".repeat(full.length);
139         });
140       }
141       for (const pat of TRAILING) {
142         working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
143           filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
144           return " ".repeat(full.length);
145         });
146       }
147
148       // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
intent).
149       let sort: Sort | undefined;
150       for (const [re, s] of SORT_PATTERNS) {
151         if (re.test(working)) {
152           sort = s;
153           break;
154         }
155       }
156
157       // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest
N").
158       let limit: number | undefined;
159       let m: RegExpMatchArray | null;
160       if ((m = working.match(/\b(\d+)\s+(longest|shortest)\b/)) || (m =
```

```
working.match(/\b(longest|shortest)\s+(\d+)\b/))) {
        const num = /^\d/.test(m[1]) ? m[1] : m[2];
        const word = /^\d/.test(m[1]) ? m[2] : m[1];
        limit = parseInt(num, 10);
        if (!sort) sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
      } else if ((m = working.match(/\b(?:top|best)\s+(\d+)\b/))) {
        limit = parseInt(m[1], 10);
        if (!sort) sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
      } else if ((m = working.match(/\b(?:first|earliest)\s+(\d+)\b/))) {
        limit = parseInt(m[1], 10);
        if (!sort) sort = { key: "start_time", dir: "asc" };
      } else if ((m = working.match(/\b(?:last|latest|newest|most recent)\s+(\d+)\b/))) {
        limit = parseInt(m[1], 10);
        if (!sort) sort = { key: "start_time", dir: "desc" };
      }

      // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
      const unavailable: Unavailable[] = [];
      const seen = new Set<string>();
      for (const [kind, re] of UNAVAILABLE_PATTERNS) {
        for (const mm of normalized.matchAll(re)) {
          const token = mm[1];
          if (!seen.has(token)) {
            seen.add(token);
            unavailable.push({ token, kind });
          }
        }
      }

      return {
        raw,
        filters,
        sort,
        limit,
        unavailable,
        recognized: filters.length > 0 || sort != null || limit != null,
      };
    }

    function rallyValue(r: RallySegment, key: SortKey): number | undefined {
      if (key === "duration") return r.duration;
      if (key === "start_time") return r.start_time;
      return r.metadata?.shot_count;
    }

    /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
    */
    export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
      return rallies
        .map((r, i) => ({ r, i }))
        .sort((a, b) => {
          const av = rallyValue(a.r, sort.key);
          const bv = rallyValue(b.r, sort.key);
          if (av == null && bv == null) return a.i - b.i;
          if (av == null) return 1;
          if (bv == null) return -1;
          if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
          return a.i - b.i; // tie ? preserve input order
        })
        .map((x) => x.r);
    }

    function matchesFilter(r: RallySegment, f: Filter): boolean {
      const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
      if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
```

```
224       switch (f.op) {
225         case ">":
226           return v > f.value;
227         case ">=":
228           return v >= f.value;
229         case "<":
230           return v < f.value;
231         case "<=":
232           return v <= f.value;
233         case "=":
234           return v === f.value;
235         default:
236           return false;
237       }
238     }
239
240     /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
241      * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
242      * so garbage input can never silently wipe a user's manual curation. The App also
guards on
243      * `q.recognized` before applying, so an empty box is a no-op end to end. */
244     export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
245       if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
246       const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
247       let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
248       if (q.limit != null) matched = matched.slice(0, q.limit);
249       return { ordered, selectedIds: matched.map((r) => r.id) };
250     }
251
252     // ?? human-readable labels (shared by the QueryBar pill and the tests) ?????????
253     const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
254
255     export function filterLabel(f: Filter): string {
256       if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
257       return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
258     }
259
260     export function sortLabel(s: Sort): string {
261       if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
262       if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
263       return s.dir === "desc" ? "most shots first" : "fewest shots first";
264     }
265
266     /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
15"]). */
267     export function summarize(q: ParsedQuery): string[] {
268       const parts: string[] = [];
269       for (const f of q.filters) parts.push(filterLabel(f));
270       if (q.sort) parts.push(sortLabel(q.sort));
271       if (q.limit != null) parts.push(`top ${q.limit}`);
272       return parts;
273     }
274
```

4. user (2026-06-20T16:37:02.169Z)

```
1       import { describe, it, expect } from "vitest";
2       import {
3         parseQuery,
4         runQuery,
5         sortRallies,
```

```
6         filterLabel,
7         sortLabel,
8         summarize,
9       } from "./query";
10      import type { RallySegment } from "../api/types";
11
12      const mk = (
13        id: number,
14        start_time: number,
15        duration: number,
16        shot_count?: number,
17      ): RallySegment => ({
18        id,
19        start_time,
20        end_time: start_time + duration,
21        duration,
22        server: null,
23        receiver: null,
24        metadata: shot_count == null ? {} : { shot_count },
25      });
26
27      // A small, varied corpus: ids ordered chronologically, durations & shot counts mixed,
28      // rally 4 deliberately has NO shot_count (tests "unknown sorts/filters honestly").
29      const RALLIES: RallySegment[] = [
30        mk(1, 12, 8, 9),
31        mk(2, 41, 6, 5),
32        mk(3, 63, 21, 19),
33        mk(4, 95, 6 /* no shot_count */),
34        mk(5, 130, 12, 11),
35        mk(6, 158, 29, 24),
36      ];
37
38      describe("parseQuery ? duration filters", () => {
39        it("'over 10s' ? duration > 10", () => {
40          expect(parseQuery("over 10s").filters).toEqual([{ field: "duration", op: ">", value:
10 }]);
41        });
42        it("accepts varied units and phrasings", () => {
43          expect(parseQuery("longer than 10 seconds").filters[0]).toEqual({ field: "duration",
op: ">", value: 10 });
44          expect(parseQuery("under 5 sec").filters[0]).toEqual({ field: "duration", op: "<",
value: 5 });
45          expect(parseQuery("at least 8s").filters[0]).toEqual({ field: "duration", op: ">=",
value: 8 });
46          expect(parseQuery("at most 8 seconds").filters[0]).toEqual({ field: "duration", op:
"<=", value: 8 });
47          expect(parseQuery("exactly 7s").filters[0]).toEqual({ field: "duration", op: "=",
value: 7 });
48        });
49        it("converts minutes to seconds", () => {
50          expect(parseQuery("over 1 minute").filters[0]).toEqual({ field: "duration", op: ">",
value: 60 });
51          expect(parseQuery("at least 1.5m").filters[0]).toEqual({ field: "duration", op:
">=", value: 90 });
52        });
53        it("handles symbols and trailing comparators", () => {
54          expect(parseQuery("? 10s").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
55          expect(parseQuery("> 10").filters[0]).toEqual({ field: "duration", op: ">", value:
10 });
56          expect(parseQuery("10s+").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
57          expect(parseQuery("10s or longer").filters[0]).toEqual({ field: "duration", op:
">=", value: 10 });
58          expect(parseQuery("8s or less").filters[0]).toEqual({ field: "duration", op: "<=",
```

```
value: 8 });
59        });
60        it("does NOT consume 'no more than' as 'more than'", () => {
61          expect(parseQuery("no more than 5s").filters[0]).toEqual({ field: "duration", op:
"<=", value: 5 });
62        });
63        it("treats a bare 'Ns' (no comparator) as not a filter", () => {
64          expect(parseQuery("10s").filters).toEqual([]);
65          expect(parseQuery("10s").recognized).toBe(false);
66        });
67      });
68
69      describe("parseQuery ? shot_count filters (noun-keyed)", () => {
70        it("routes the 'shots' noun to shot_count, not duration", () => {
71          expect(parseQuery("over 10 shots").filters[0]).toEqual({ field: "shot_count", op:
">", value: 10 });
72          expect(parseQuery("at least 8 shots").filters[0]).toEqual({ field: "shot_count", op:
">=", value: 8 });
73          expect(parseQuery("fewer than 5 shots").filters[0]).toEqual({ field: "shot_count",
op: "<", value: 5 });
74          expect(parseQuery("10+ shots").filters[0]).toEqual({ field: "shot_count", op: ">=",
value: 10 });
75        });
76      });
77
78      describe("parseQuery ? sort", () => {
79        const cases: Array<[string, ReturnType<typeof parseQuery>["sort"]]> = [
80          ["longest first", { key: "duration", dir: "desc" }],
81          ["shortest", { key: "duration", dir: "asc" }],
82          ["newest first", { key: "start_time", dir: "desc" }],
83          ["oldest first", { key: "start_time", dir: "asc" }],
84          ["in order", { key: "start_time", dir: "asc" }],
85          ["most shots", { key: "shot_count", dir: "desc" }],
86          ["fewest shots", { key: "shot_count", dir: "asc" }],
87        ];
88        it.each(cases)("'%s' ? %o", (input, expected) => {
89          expect(parseQuery(input).sort).toEqual(expected);
90        });
91      });
92
93      describe("parseQuery ? limit (and its implied sort)", () => {
94        it("'top 15' ? limit 15, sort longest-first", () => {
95          const q = parseQuery("top 15");
96          expect(q.limit).toBe(15);
97          expect(q.sort).toEqual({ key: "duration", dir: "desc" });
98        });
99        it("'first 10' ? limit 10, sort oldest-first", () => {
100         const q = parseQuery("first 10");
101         expect(q.limit).toBe(10);
102         expect(q.sort).toEqual({ key: "start_time", dir: "asc" });
103       });
104       it("'15 longest' / '5 shortest' carry both limit and sort", () => {
105         expect(parseQuery("15 longest")).toMatchObject({ limit: 15, sort: { key: "duration",
dir: "desc" } });
106         expect(parseQuery("5 shortest")).toMatchObject({ limit: 5, sort: { key: "duration",
dir: "asc" } });
107       });
108       it("an explicit sort wins over the limit's implied one ('top 15 longest')", () => {
109         const q = parseQuery("top 15 longest");
110         expect(q.limit).toBe(15);
111         expect(q.sort).toEqual({ key: "duration", dir: "desc" });
112       });
113       it("combines a filter with a limit ('over 10s top 5')", () => {
114         const q = parseQuery("over 10s top 5");
115         expect(q.filters).toEqual([{ field: "duration", op: ">", value: 10 }]);
```

```
116         expect(q.limit).toBe(5);
117       });
118     });
119
120     describe("parseQuery ? unavailable concepts are flagged, never faked", () => {
121       it("flags shot-type words while still applying the available clause", () => {
122         const q = parseQuery("smashes over 10s");
123         expect(q.filters).toEqual([{ field: "duration", op: ">", value: 10 }]); // honoured
124         expect(q.unavailable).toEqual([{ token: "smashes", kind: "shot-type" }]); // flagged
125       });
126       it("flags player and score concepts", () => {
127         expect(parseQuery("opponent rallies").unavailable).toContainEqual({ token:
"opponent", kind: "player" });
128         expect(parseQuery("game point").unavailable).toContainEqual({ token: "game point",
kind: "score" });
129       });
130       it("does not confuse 'shots' (count) with a shot-type", () => {
131         expect(parseQuery("over 10 shots").unavailable).toEqual([]);
132       });
133       it("a pure unavailable query is not recognized as actionable", () => {
134         const q = parseQuery("rallies with drops");
135         expect(q.recognized).toBe(false);
136         expect(q.unavailable).toContainEqual({ token: "drops", kind: "shot-type" });
137       });
138     });
139
140     describe("parseQuery ? unrecognized / empty input", () => {
141       it("returns recognized=false with nothing to apply", () => {
142         for (const s of ["", "   ", "hello there", "asdf"]) {
143           const q = parseQuery(s);
144           expect(q.recognized).toBe(false);
145           expect(q.filters).toEqual([]);
146           expect(q.sort).toBeUndefined();
147           expect(q.limit).toBeUndefined();
148         }
149       });
150     });
151
152     describe("sortRallies", () => {
153       it("sorts by duration desc / asc, stably", () => {
154         expect(sortRallies(RALLIES, { key: "duration", dir: "desc" }).map((r) =>
r.id)).toEqual([6, 3, 5, 1, 2, 4]);
155         expect(sortRallies(RALLIES, { key: "duration", dir: "asc" }).map((r) =>
r.id)).toEqual([2, 4, 1, 5, 3, 6]);
156       });
157       it("puts rallies with a missing field LAST regardless of direction", () => {
158         // rally 4 has no shot_count ? last whether asc or desc.
159         expect(sortRallies(RALLIES, { key: "shot_count", dir: "desc" }).map((r) =>
r.id)).toEqual([6, 3, 5, 1, 2, 4]);
160         expect(sortRallies(RALLIES, { key: "shot_count", dir: "asc" }).map((r) =>
r.id)).toEqual([2, 1, 5, 3, 6, 4]);
161       });
162       it("does not mutate the input array", () => {
163         const ids = RALLIES.map((r) => r.id);
164         sortRallies(RALLIES, { key: "duration", dir: "desc" });
165         expect(RALLIES.map((r) => r.id)).toEqual(ids);
166       });
167     });
168
169     describe("runQuery ? end to end", () => {
170       it("selects every rally over 10s, in chronological order (no sort)", () => {
171         const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("over 10s"));
172         expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]); // unsorted ? input
order
173         expect(selectedIds).toEqual([3, 5, 6]); // 21s, 12s, 29s
```

```
174       });
175       it("'top 3 longest' sorts all and selects the 3 longest", () => {
176         const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("top 3 longest"));
177         expect(ordered.map((r) => r.id)).toEqual([6, 3, 5, 1, 2, 4]); // all, longest-first
178         expect(selectedIds).toEqual([6, 3, 5]);
179       });
180       it("a shot filter excludes rallies that have no shot_count", () => {
181         const { selectedIds } = runQuery(RALLIES, parseQuery("at least 10 shots"));
182         expect(selectedIds).toEqual([3, 5, 6]); // rally 4 (no shot_count) excluded
183       });
184       it("filter + limit interact (longest 2 of the >10s rallies)", () => {
185         const { selectedIds } = runQuery(RALLIES, parseQuery("over 10s top 2"));
186         expect(selectedIds).toEqual([6, 3]); // sorted longest-first then capped at 2
187       });
188       it("an unrecognized query selects nothing and leaves order untouched", () => {
189         const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("smashes only"));
190         expect(selectedIds).toEqual([]);
191         expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]);
192       });
193     });
194
195     describe("labels", () => {
196       it("filterLabel renders honest field + symbol + unit", () => {
197         expect(filterLabel({ field: "duration", op: ">", value: 10 })).toBe("length > 10s");
198         expect(filterLabel({ field: "shot_count", op: ">=", value: 8 })).toBe("shots ? 8");
199       });
200       it("sortLabel reads naturally for every key/dir", () => {
201         expect(sortLabel({ key: "duration", dir: "desc" })).toBe("longest first");
202         expect(sortLabel({ key: "duration", dir: "asc" })).toBe("shortest first");
203         expect(sortLabel({ key: "start_time", dir: "desc" })).toBe("newest first");
204         expect(sortLabel({ key: "start_time", dir: "asc" })).toBe("oldest first");
205         expect(sortLabel({ key: "shot_count", dir: "desc" })).toBe("most shots first");
206         expect(sortLabel({ key: "shot_count", dir: "asc" })).toBe("fewest shots first");
207       });
208       it("summarize lists filter, sort and limit clauses", () => {
209         expect(summarize(parseQuery("over 10s top 5"))).toEqual(["length > 10s", "longest
first", "top 5"]);
210         expect(summarize(parseQuery("hello"))).toEqual([]);
211       });
212     });
213
```

5. assistant (2026-06-20T16:37:13.522Z)

Now I have the full picture. Let me trace the finding carefully.

Verifying finding #1's premise (the mislabel): The `summarize()` function at line 271 unconditionally renders any limit as `top ${q.limit}`. So `parseQuery("first 10")` produces `limit: 10, sort: {key:"start_time", dir:"asc"}`, and `summarize()` would output `["oldest first", "top 10"]`. The pill says "top 10" even though the query was "first 10" / oldest-first. That mislabel is real in the code.

Verifying the test-gap claim itself: Scanning every `summarize(` call in the test file:
- Line 209: `summarize(parseQuery("over 10s top 5"))` ? `["length > 10s", "longest first", "top 5"]` ? the `top`/duration-desc path.
- Line 210: `summarize(parseQuery("hello"))` ? `[]` ? the empty path.

Those are the only two `summarize` assertions. Confirmed: there is **no** `summarize` test for `first N`, `last N`, or `N shortest`. The only limit-bearing `summarize` case is the `top` case, which happens to be the one input shape where the `top ${limit}` label is coincidentally correct. So the test suite would stay green even while the pill prints "top 10" for "first 10".

The finding's evidence is accurate (line 209 quoted exactly, only-limit-summarize-case confirmed), the reasoning is sound (the test coverage shape is exactly why the mislabel escapes), and the severity is appropriately `minor` (a test-coverage gap, not a runtime bug on

its own; the user-facing mislabel is finding #1's blocker/major).

One nuance: this is a *test-gap* finding dependent on finding #1 being the actual defect. The test gap is genuine and verifiable independently ? the suite gives false confidence that the pill is honest for all query shapes. I'll confirm it as real.

6. user (2026-06-20T16:37:22.059Z)

Structured output provided successfully

7. assistant (2026-06-20T16:37:26.439Z)

Verified the finding as real. The test gap is genuine: `summarize()` has only two assertions in the entire suite (query.test.ts lines 209 and 210), and the single limit-bearing case uses the one input shape ("top 5") where the hardcoded `top ${limit}` label at query.ts:271 is coincidentally correct ? so the mislabel of "first N"/"last N"/"N shortest" as "top N" escapes the suite entirely.